LANGCHAIN DEVELOPMENT

Understanding LangChain

LangChain is an open-source framework for **building AI applications powered by large language models**. It provides standard interfaces for models, prompts, memory, vector stores, and chains, letting developers easily **"chain together interoperable components"** in AI workflows. In practice, LangChain helps connect LLMs (like GPT-4/5, Claude, etc.) with data, tools, and state management, enabling rapid prototyping of complex pipelines.

- **Key concepts:** modular components (LLMs, prompts, tools, memory, vector DBs) orchestrated by **chains** or **agents**. LangChain abstracts providers and lets you swap models or backends without rewriting logic.
- **Example (Python):**

python

Copy

```
from langchain.chat_models import ChatOpenAI

model = ChatOpenAI(model="gpt-4")

response = model.invoke("Hello, world!")  # call the LLM directly

print(response.content)
```

- **Common pitfalls:** overcomplicating with many components; ignoring context limits (long conversations). The flexibility can lead to confusion if components aren't tested in isolation.
- **Study tips:** Practice simple pipelines first (e.g. a single prompt → model → output). Learn one component at a time (e.g. prompts, then a chain with memory). Keep the LangChain docs handy, as they define core abstractions.
- **Flashcards:**
    - *Q:* What is LangChain? *A:* A framework for building LLM-powered AI apps by chaining components.
    - *Q:* Why use LangChain? *A:* For modular AI workflows, model/tool integration, and rapid prototyping.
    - *Q:* Name LangChain's core components. *A:* Models, Prompts, Memory, Chains, Agents, Tools, Vector Stores.

Using Generative AI Models

Generative AI models (LLMs) are **"powerful AI tools that can interpret and generate text like humans"**. In LangChain, models are the "reasoning engine" of your app or agent. You interact with models either **standalone** (direct calls) or **within agents/chains**. LangChain provides unified classes (e.g. ChatOpenAI, ChatAnthropic) to initialize any supported provider easily. Because models vary (e.g. some excel at reasoning, others at longer context), LangChain's abstractions let you swap providers for benchmarking.

- **Key concepts:** LLMs understand prompts and produce text, but also can **call tools**, output structured data, and handle multimodal input (images, etc.). LangChain's standard interface makes it easy to experiment with different models (OpenAI, Anthropic, HuggingFace, etc.).
- **Example (Python):**

python

Copy

```
from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-5.4")

result = model.invoke("Translate 'hello' into Spanish.")

print(result.content)  # e.g. "hola"
```

- **Common pitfalls:** Models have **context limits**; exceeding them causes errors or dropped text. Also be mindful of **hallucinations** (fabricated answers). Ensure API keys, rate limits and usage costs are managed. Different models have different pricing and latency.
- **Study tips:** Start with a small prompt and inspect the raw output. Try a few different models for the same task to see differences. Use the same interface for agents or direct calls to stay consistent.
- **Flashcards:**
    - *Q:* How to call a model in LangChain? *A:* Use classes like ChatOpenAI or init_chat_model("provider:model").
    - *Q:* What does "structured output" mean? *A:* Constraining model responses to a defined format (e.g. JSON).
    - *Q:* Why might you switch models in LangChain? *A:* For different capabilities (longer context, cheaper cost) using the same interface.

Creating and Managing Prompts

A **prompt** is the input text to an LLM. In LangChain, prompts are usually **templates** with placeholders for dynamic values. Prompt engineering involves designing these templates (and few-shot examples) to get the LLM to do the desired task. LangChain offers two main classes: PromptTemplate for single-text prompts and ChatPromptTemplate for multi-role (system/user/assistant) conversations. Templates can use Python f-string ({var}) syntax for simple cases, or Mustache ({{var}}) for loops/conditionals in more complex prompts.

- **Key concepts:** A *PromptTemplate* defines a string with variables:

python

Copy

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template("Write a summary of: {topic}")

text = prompt.format(topic="LangChain uses")
```

A *ChatPromptTemplate* defines a sequence of messages (system, human, AI) with placeholders. LangChain also supports **output parsers** to extract structured answers.

- **Example (Python):** Creating a chat prompt template:

python

Copy

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([

  ("system", "You are a helpful assistant."),

  ("user", "Explain {concept} in simple terms."),

])

result = (prompt | model).invoke({"concept": "LangChain"})

print(result)  # e.g., "LangChain is a framework that helps build..."
```

- **Common pitfalls:** Forgetting to format variables or misnaming them. Templates can produce errors if placeholders are missing. Prompt injection is a risk (users manipulating prompts). Avoid overly long prompts or too many examples (exceeds context).

- **Study tips:** Test prompts iteratively. Print the formatted prompt before sending it to the model to ensure it looks right. Compare single-shot vs few-shot. Learn output parsing to handle model replies reliably.
- **Flashcards:**
    - *Q:* What is a PromptTemplate? *A:* A reusable string template with placeholders for variables.
    - *Q:* When to use ChatPromptTemplate? *A:* For multi-turn chat where roles (system, user, assistant) are specified.
    - *Q:* What syntax does PromptTemplate support? *A:* Python-style f-strings ({var}) for simple cases.

Learning about Memory

Memory in LangChain lets an agent or chain *remember* past interactions or knowledge. Memory can be **short-term** (conversation history within a session) or **long-term** (persistent facts across sessions). Short-term memory is typically just the recent chat history kept in the context window. Long-term memory uses external stores (often vector databases) to recall relevant info (like user profile or facts) later. LangChain provides classes like ConversationBufferMemory (stores all messages) and VectorStoreRetrieverMemory (stores memory embeddings).

- **Key concepts:** Memory is vital for multi-turn dialogs and personalization. For example, a bot can **remember a user's name or past answers**. Long-term memory might store user preferences as embeddings so the model can query it when needed.
- **Example (Python):** Using a conversation memory in a chain:

python

Copy

```
from langchain.memory import ConversationBufferMemory

from langchain import ConversationChain

memory = ConversationBufferMemory()

chain = ConversationChain(llm=ChatOpenAI(model="gpt-4"), memory=memory)

chain("Hello!")      # Bot responds

chain("Who am I?")    # Bot can answer based on earlier memory
```

Or vector memory: store embeddings in Pinecone/Chroma and retrieve in queries.

- **Common pitfalls:** Context can overflow if memory grows too large. LLMs may get “distracted” by irrelevant old details. Overwriting memory with noisy data leads to confusion. Also beware of privacy: never store sensitive data in memory stores.
- **Study tips:** Use summarization memory (e.g. ConversationSummaryMemory) to condense long chats. Limit memory by keywords or recency. Practice retrieving relevant memory by asking the model to recall specific facts.
- **Flashcards:**
    - *Q:* What is short-term memory? *A:* Conversation history in the current thread.
    - *Q:* What is long-term memory? *A:* Persistent knowledge stored across sessions, often in a database.
    - *Q:* What does ConversationBufferMemory do? *A:* Keeps all past user/AI messages in a buffer.

Deciphering Chains

A **chain** is a sequence of operations (prompts, model calls, functions) linked together to accomplish a task. Modern LangChain uses the **LangChain Expression Language (LCEL)** with the pipe operator (|) to compose chains. In a simple chain, you might take user input, format it with a prompt, pass it to a model, then parse the output. Complex chains can have branches, loops, or conditional steps. Chains help modularize logic and reuse components.

User Input

Prompt Template

LLM Model

Output

Show code

- **Key concepts:** Chains can be **sequential** (step1→step2→...), **parallel** (multiple pipelines run on same input), or **conditional** (routing based on content). Common patterns include *Prompt+Model+Parser* (format input, call LLM, parse text). Legacy classes like LLMChain or SimpleSequentialChain existed, but now the pipe syntax is recommended.
- **Example (Python):** Chaining a prompt, model, and parser:

python

Copy

```
from langchain_core.prompts import ChatPromptTemplate

from langchain_openai import ChatOpenAI

from langchain_core.output_parsers import StrOutputParser


# Define components

prompt = ChatPromptTemplate.from_template("What is the capital of {country}?")

model = ChatOpenAI(model="gpt-4")

parser = StrOutputParser()


# Compose chain

chain = prompt | model | parser

result = chain.invoke({"country": "France"})

print(result)  # e.g. "Paris"
```

- **Common pitfalls:** Long chains can be hard to debug, and each added step adds latency and potential failure points. Ensure each sub-chain works independently. Be cautious with parallel chains that might hit rate limits on models.
- **Study tips:** Draw out the chain steps on paper or with a flowchart (as above). Test chains end-to-end and inspect intermediate outputs. Use logging callbacks in LangChain to trace values through the chain.
- **Flashcards:**
    - *Q:* How do you compose a chain in modern LangChain? *A:* Use the pipe operator (|) to connect runnables (prompt, model, parser).
    - *Q:* What is a common chain pattern? *A:* Prompt → Model → Output Parser.
    - *Q:* What happens if you provide no tools to an agent? *A:* The agent is just a single LLM node without tool-calling.

Building Intelligent Agents

An **agent** in LangChain is a system that uses an LLM to make decisions and call external tools to perform tasks. Agents "**combine language models with tools**" so they can reason about what action to take next. Typically, an agent runs in a loop: it looks at the

user’s query and context, decides which (if any) tool to invoke, executes that tool, feeds results back into the model, and repeats until it outputs a final answer. Agents are ideal for tasks requiring dynamic information retrieval, computation, or action (like answering questions via web search).

ToolChain

Conversation

calls tool

final answer

System Prompt

LLM Model

User Query

Decide Action

Tool Function

LLM Answer

Show code

- **Key concepts:** Tools are external functions (APIs, code) that the agent can use. For example, a calculator or search API can be tools. LangChain lets you define tools easily with @tool or by passing callables. You then create an agent by supplying a model and a list of tools to create_agent. Agents may have static tools (fixed set) or dynamic tools (selected at runtime via middleware).
- **Example (Python):** A simple agent with a calculator tool:

python

Copy

```
from langchain.agents import create_agent

from langchain.tools import tool

from langchain_openai import ChatOpenAI


@tool

def calc(expr: str) -> str:

    """Perform arithmetic."""
```

```
    return str(eval(expr))


model = ChatOpenAI(model="gpt-5.4")

agent = create_agent(model, tools=[calc])

result = agent.invoke({"input": "What is 12 * 7?"})

print(result)  # e.g. "84"
```

- **Common pitfalls:** Agents can be unpredictable: the model might choose the wrong tool or loop indefinitely. It may "hallucinate" using tools that don't exist. Designing clear tool descriptions and a good **system prompt** is crucial. Also, each tool call adds latency and cost, so only include useful tools. Ensure you handle tool errors or unexpected outputs gracefully (LangChain supports retry logic).
- **Study tips:** Start with one tool and build up. Inspect the agent's reasoning by logging its steps. Use a permissive "max iterations" to catch loops. Keep the system prompt concise but instructive.
- **Flashcards:**
    - *Q:* What do agents do? *A:* Use an LLM + tools to iteratively solve tasks.
    - *Q:* How to create an agent in LangChain? *A:* Use create_agent(model, tools=[...]).
    - *Q:* What is a Tool in LangChain? *A:* A function (often decorated with @tool) that the LLM can call to perform an action.

Exploring Indexes and Vector Databases

Vector databases (vector stores) are specialized systems for storing and querying embeddings (high-dimensional vectors). They enable **semantic search** or Retrieval-Augmented Generation (RAG) by finding items whose embeddings are similar to a query embedding. LangChain integrates with many vector stores (Pinecone, Weaviate, Qdrant, Chroma, etc.) via uniform APIs. Typically, you embed your documents into vectors, upsert them into an index, and then perform a similarity search when answering a query. This is crucial for tasks like chatbots that "know" a knowledge base.

- **Key concepts:** An **index** or **vector store** holds items (with payload) and supports similarity queries (e.g. nearest neighbors). Example workflows:
    1. **Embed** documents (e.g. using OpenAI or HuggingFace embeddings).
    2. **Upsert** vectors into a store (with IDs and metadata).

3. **Query** the store with a query vector to retrieve relevant items. LangChain's VectorStore classes (e.g. Pinecone, Chroma) wrap these steps. A special memory (like VectorStoreRetrieverMemory) can also use a vector DB for long-term memory.

- **Example (Python):** Using Pinecone via LangChain:

python

Copy

```
import pinecone

from langchain.vectorstores import Pinecone

from langchain.embeddings import OpenAIEmbeddings


pinecone.init(api_key="PINECONE_API_KEY", environment="us-west1-gcp")

index = pinecone.Index("myindex")

store = Pinecone(index, OpenAIEmbeddings(), "text")  # "text" is metadata key

docs = ["LangChain is great for LLM apps.", "Vector DBs store embeddings."]

store.add_texts(docs, metadatas=[{"text": d} for d in docs])

results = store.similarity_search("How do vector DBs work?", k=2)

print([doc.page_content for doc in results])
```

- **Common pitfalls:** Each vector DB has limits on vector dimensionality and index size. Exceeding capacity or not normalizing vectors can degrade search quality. Watch out for cold-start latency (first queries may be slower) and ensure you provision enough resources (especially for cloud SaaS services).
- **Study tips:** Experiment with a small local store (like Chroma in-memory) first. Visualize embedding distances for some test queries to build intuition. Compare different DBs by measuring query latency and result quality on sample data.
- **Flashcards:**
    - *Q:* What is a vector store? *A:* A database for storing and querying high-dimensional embeddings (for similarity search).
    - *Q:* Why use a vector DB in an LLM app? *A:* To retrieve relevant context (knowledge) for RAG or memory, based on semantic similarity.

- *Q:* How do you perform a similarity search in Pinecone? *A:* Upsert vectors then call index.query(vector=query_vec, top_k=k) (or via LangChain's similarity_search).

Vector Database Comparison

DB	Key Features	Scalability	Latency	Pricing Model	Language/API	Use-Cases
Pinecone	Fully-managed, cloud-native; automatic sharding and scaling; real-time indexing	Very high (serverless cloud)	Low (ms)	Paid SaaS (tiered/usage)	Python, REST API	Semantic sear RAG, recommendat
Weaviate	Open-source (BSD-3-Clause); supports HNSW & flat indexes; vector+metadata hybrid queries	Horizontal scaling (sharding)	Low (ms)	Free OSS or paid Cloud / self-host	Python, JS, Go, Java (GraphQL API)	Semantic sear NLP, RAG
Qdrant	Open-source (Apache-2.0); Rust core for speed; advanced filtering, scalar storage	Cluster mode; free and managed cloud	Sub-ms to low-ms	Free OSS; Qdrant Cloud with free tier	Python, Go, JS, .NET, Java, etc.	Search at scal filtering, RAG
Chroma	Open-source (Apache-2.0) optimized for developer use; object-storage backend	Scales with data (auto-tiering)	Low (tens of ms)	Free OSS; "serverless" cloud credits	Python, TypeScript, Rust	NLP apps, RAG hybrid search
Milvus	Open-source; GPU support; C++ core for high performance	Massive (clusters, GPU)	Very low (sub-ms)	Free OSS or managed cloud (Zilliz)	Python, Go, Java	Large-scale embeddings, C audio
Faiss	Library (not a service); very fast ANN search (Facebook)	In-memory or disk index (limited)	Very low (sub-ms)	Free (library)	Python, C++	Research, loca similarity sear

(Sources: Pinecone docs; Weaviate overview; Qdrant docs; Chroma site.)

Key takeaways: Pinecone is easy and fast but proprietary. Weaviate and Qdrant are open-source for full control. Chroma balances developer friendliness (Python-first) with scalability via object storage. Latencies are generally in milliseconds; use-case suitability varies (see table).